UT4

VEGA MORENO
MÚRIEL CADÓRNIGA

OPTIMIZACIÓN Y DOCUMENTACIÓN

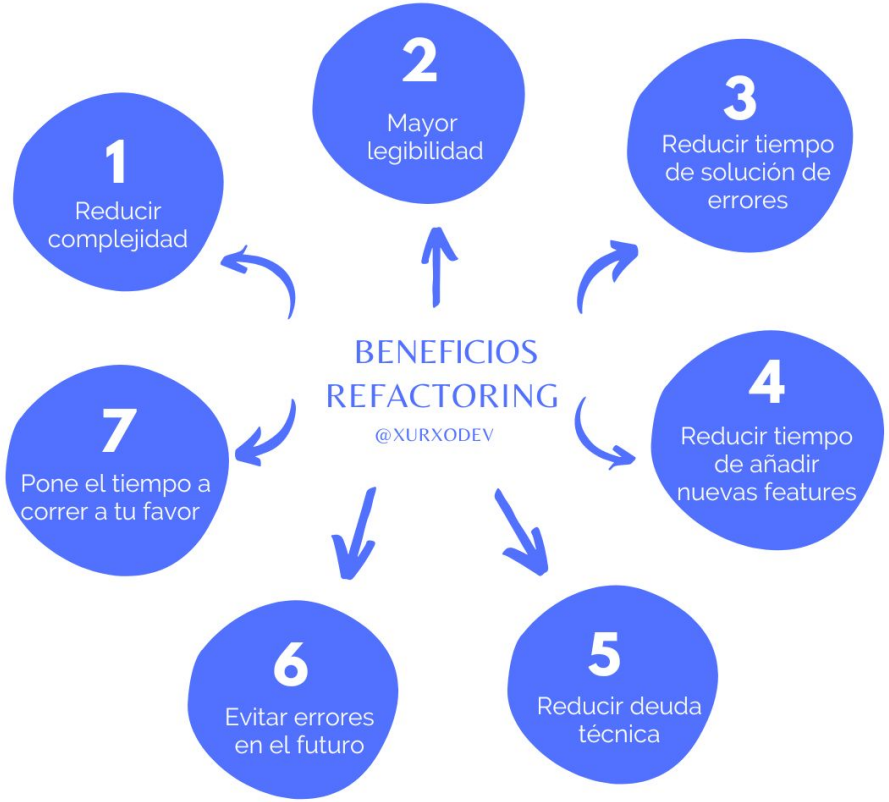
ÍNDICE

1. REFACTORIZACIÓN
   - ¿Qué es?
   - Limitaciones
   - Patrones de refactorización
   - Refactorización y pruebas
   - Herramientas de ayuda
2. CONTROL DE VERSIONES
   - Desarrollos colectivos
   - Control de cambios
   - Herramientas de control de versión
   - Utilidad en proyectos reales
   - Características principales
   - Estructura del sistema
   - Operaciones básicas
   - Operaciones avanzadas
3. DOCUMENTACIÓN DEL SOFTWARE
   - Uso de comentarios
   - Buenas prácticas
   - Documentación automática
   - Alternativas de documentación

1.1 ¿Qué es la refactorización?

Se trata de **reorganizar, limpiar y mejorar** el código fuente manteniendo exactamente el mismo comportamiento externo. Por lo que **no altera el resultado que es recibido** por el usuario, si no la parte interna para optimizarla.

El programa funciona igual, pero el código se **entiende mejor**, es más **fácil de modificar** y tiene **menos errores potenciales**.



1.1 ¿Qué es la refactorización?

Esto se puede ver cuando en un programa, por ejemplo, las variables no tienen **nombres prácticos** que den a entender su propósito, por lo que se les **cambia**.

```
int a=2;
double b=a+(a*0.21);

System.out.println(b+" euros");
```

Ejemplo para calcular el **precio del pan + IVA**

```
int precioPanBase=2;
double precioPanIVA=precioPanBase+(precioPanBase*0.21);

System.out.println("El precio base del pan son "+precioPanBase+" euros, más IVA son "+precioPanIVA);
```

1.2 Limitaciones

★ **No añade funcionalidades nuevas**

No añade cosas nuevas, si no que **mejora lo que ya existe**. Por lo que mediante refactorización **no se pueden añadir funciones**, eso sería desarrollo.

★ **Riesgos si no existen pruebas**

Sin pruebas, puedes **romper algo sin darte cuenta** y luego el **error aparece mucho más tarde**. Por ejemplo, cambiar un método creyendo que no va a tener repercusiones, pero otro método dependía de ese por lo que se produce un fallo.

★ **Tiempo y coste**

Refactorizar **requiere tiempo** y sus cambios el cliente no los aprecia, por lo que no siempre es conveniente hacerlo, **solamente si aporta un valor real.**

★ **No corrige errores de diseño profundo**

Si el diseño del programa tiene **errores demasiado graves**, refactorizar solamente no es suficiente, se puede **necesitar rediseñar y reescribir.**

1.3 Patrones de refactorización

- ## Renombrar variables y métodos

  Es uno de los más **comunes y útiles**. Funciona dando nombres más **claros y prácticos**.

- ## Extraer método

  Si tienes un método **demasiado largo**, puedes **dividir su interior en diferentes métodos** más simples y pequeños aparte del método, y luego importarlos a este. De esta forma el código queda más **claro, organizado y reutilizable.**

```
public void procesarPedido(){
    // validar datos
    // calcular total
    // aplicar descuentos
    // guardar pedido
}
```

```
public void procesarPedido(){
    validarDatos();
    calcularTotal();
    aplicarDescuentos();
    guardarPedido();
}
```

1.3 Patrones de refactorización

- ## Mover método o clase

  Funciona cuando un método o clase se encuentra en un **lugar que no le corresponde**. Mejora la **cohesión** del diseño. Por ejemplo, un método suma() se encuentre en la clase Pizarra, donde no debería, por lo que se **mueve** a la clase Operaciones.

- ## Eliminar código duplicado

  Si en el código hay partes **redudnantes** o directamente duplicadas, las **elimina** para aumentar la **optimización del diseño** y **deshacerse de futuros errores duplicados**.

- ## Simplificación de condicionales

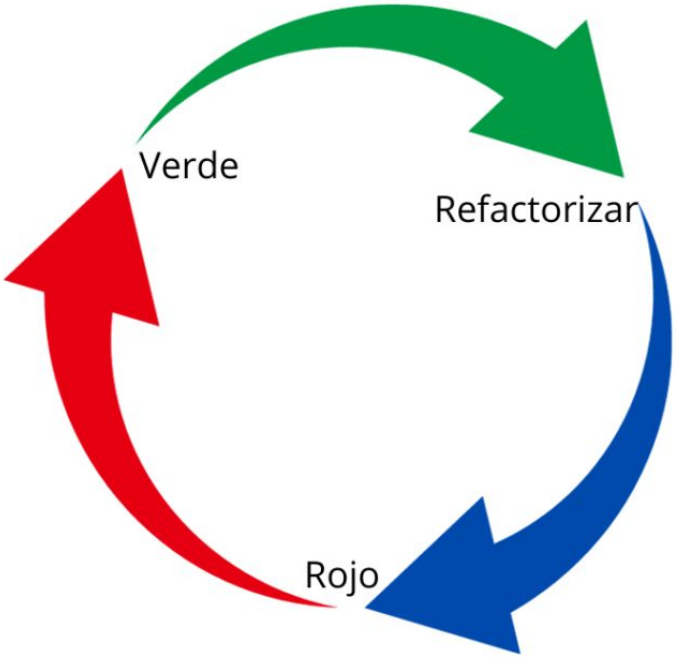
  Este patrón **simplifica condicionales** que son **complicados de más**, cuando podrían no serlo. Funciona sobre todo con los booleanos, ya que se basan solamente en la condición de verdadero o falso y se pueden simplificar mucho.

1.4 Refactorización y pruebas

Durante la refactorización, el uso de **pruebas** para comprobar el funcionamiento del código es muy importante:

Las pruebas **automatizadas** son pruebas realizadas que garantizan que el programa **siga funcionando igual** y que los **cambios no rompan nada**. Si una prueba falla tras la refactorización, es porque algo se ha roto.

Las pruebas **unitarias** son pruebas **rápidas** que sirven como **apoyo**, al probar **métodos más pequeños**. Por ejemplo: un método de prueba que compruebe que el valor real que tiene que dar una operación, y el valor que realmente calcula el código, sean iguales.



1.5 Herramientas de ayuda

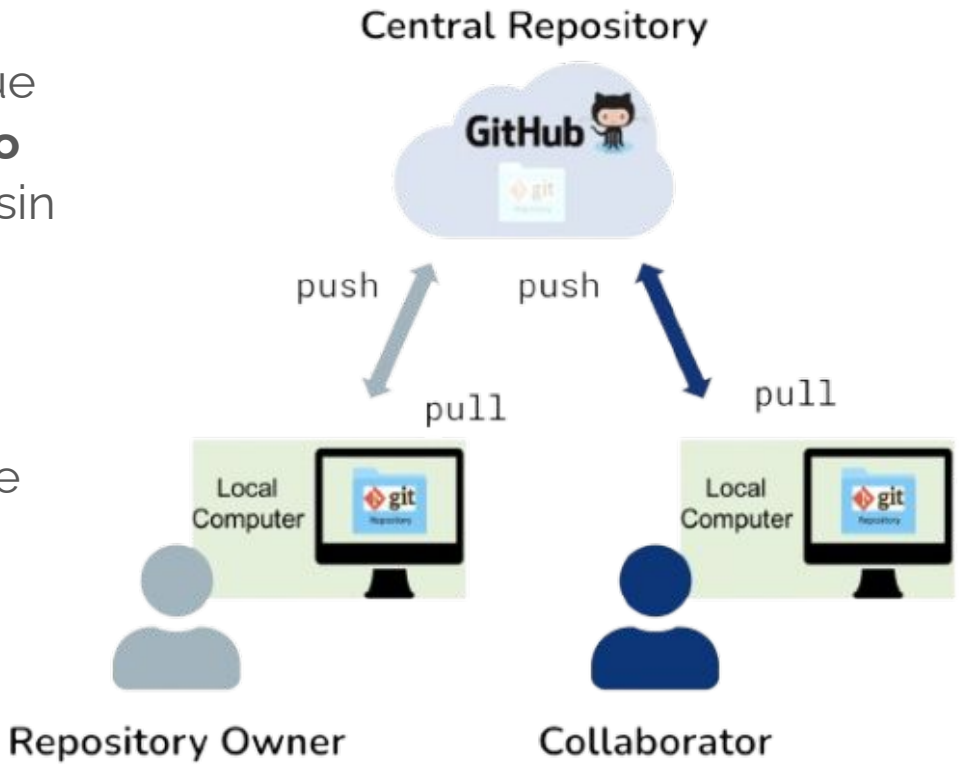
Los IDEs modernos vienen con **funciones automáticas** que facilitan la refactorización:

1. NetBeans: En refactorizar, aporta: **renombrar, extraer método, y mover.**
2. Eclipse: Usa atajos de teclado que hacen: **Alt + Shift + R □ renombrar, Alt + Shift + M □ extraer método**, y **Alt + Shift + V □ extraer variable**.
3. IntelliJ: Da **sugerencias** automáticas de refactorización, detecciones de problemas con **"code smells"** y lleva a cabo **refactorizaciones complejas seguras**.
4. VS Code: Lleva a cabo **renombrado inteligente** y **refactorización básica pero efectiva** con soporte mediante extensiones.

2. Control de versiones - Desarrollos colectivos

El control de versiones permite que **varias personas trabajen al mismo tiempo** sobre un mismo proyecto sin interferirse entre sí.

Cada miembro puede realizar cambios de forma **segura y coordinada**, evitando la pérdida de información.



2. Control de versiones - Control de cambios

Todas las **modificaciones** realizadas en el proyecto quedan **registradas**. Esto permite saber **qué** se ha cambiado, **cuándo y quién** lo ha hecho, facilitando el seguimiento y la corrección de errores.

```
$ git log
commit 6cf7d1a76c51a2875e14d19ce1ecb78017b4fd94 (HEAD)
Author: [redacted]
Date:   Fri Apr 13 10:14:49 2018 +0200

    Hemos renombrado el archivo holamundo.js para llamarlo adiosmundo.js

commit 132faa2bfb1fc295335f916cb14e8f4a6f3de83b
Author: [redacted]
Date:   Wed Apr 11 11:35:27 2018 +0200

    Hemos realizado cambios en datos.txt, holamundo.css, holamundo.html, y holam
undo.js

commit ba6eef8f64220e2f7055379489614569e5d78551
Author: [redacted]
Date:   Thu Mar 29 19:44:00 2018 +0200

    Hemos modificado holamundo.html

commit ffd325d1858c335d0b712f7c834c7ef8075f332f
Author: [redacted]
Date:   Thu Mar 29 18:56:46 2018 +0200
```

2. Control de versiones - Herramientas

Existen diferentes **herramientas para gestionar versiones**. **Git** es la más utilizada actualmente, aunque también existen otras como **SVN** o **Mercurial**, usadas en determinados entornos.

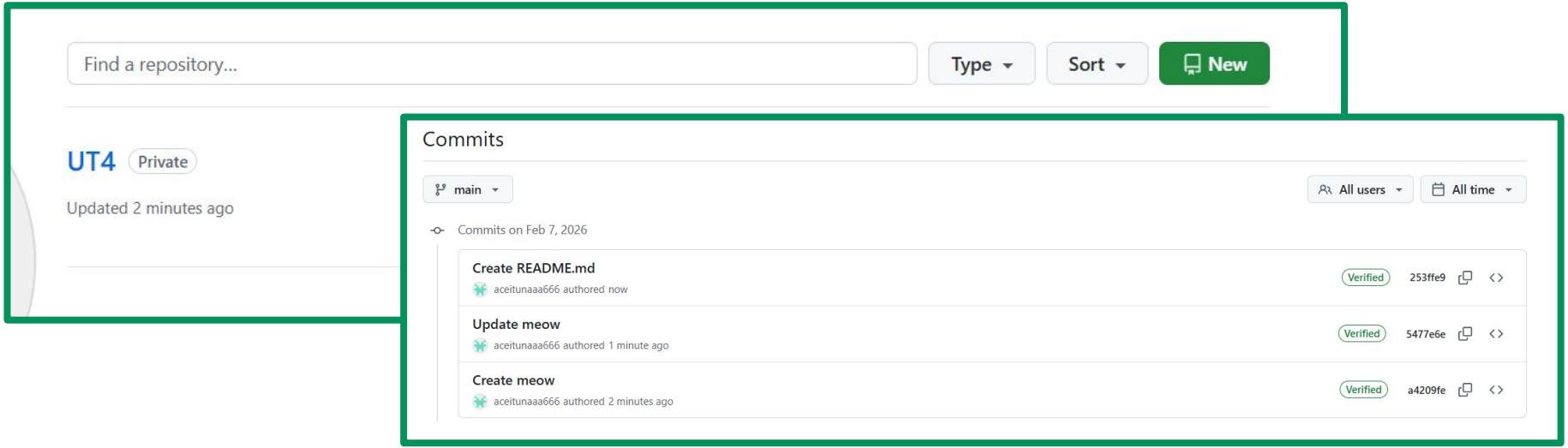
MERCURIAL

GIT

SVN

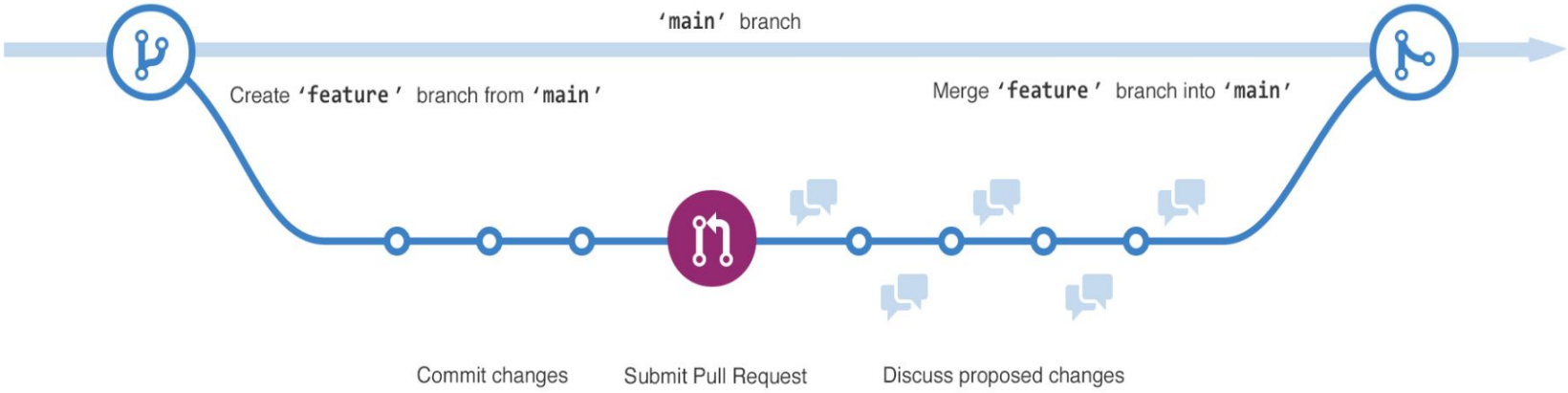
2. Control de versiones - Utilidad en proyectos reales

En proyectos profesionales, el control de versiones es imprescindible para mantener el **orden** del código, trabajar en **equipo** y asegurar **copias de seguridad** constantes del proyecto.



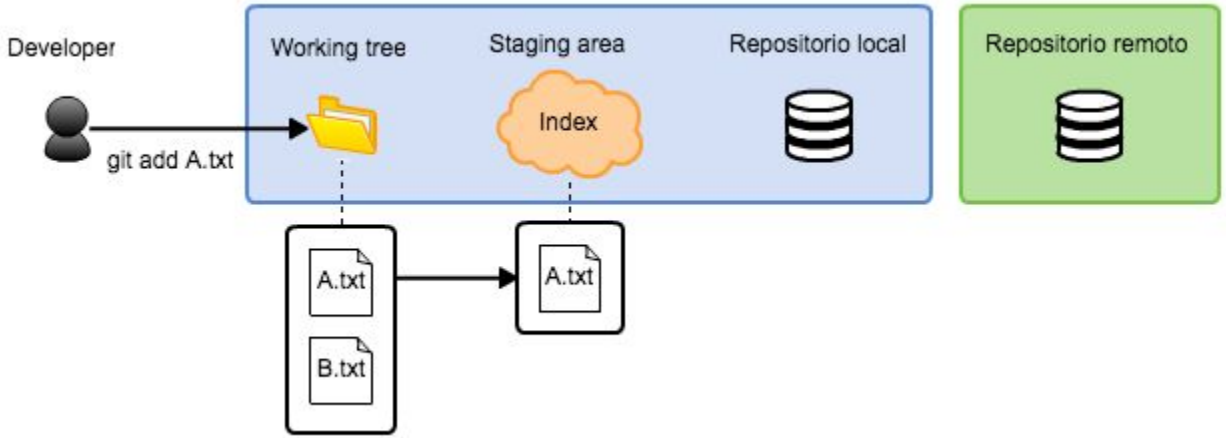
2. Control de versiones - Características principales

Los sistemas de control de versiones permiten mantener un **historial completo del proyecto**, crear **ramas para trabajar en nuevas funcionalidades** y **unir cambios** mediante fusiones, resolviendo posibles conflictos.



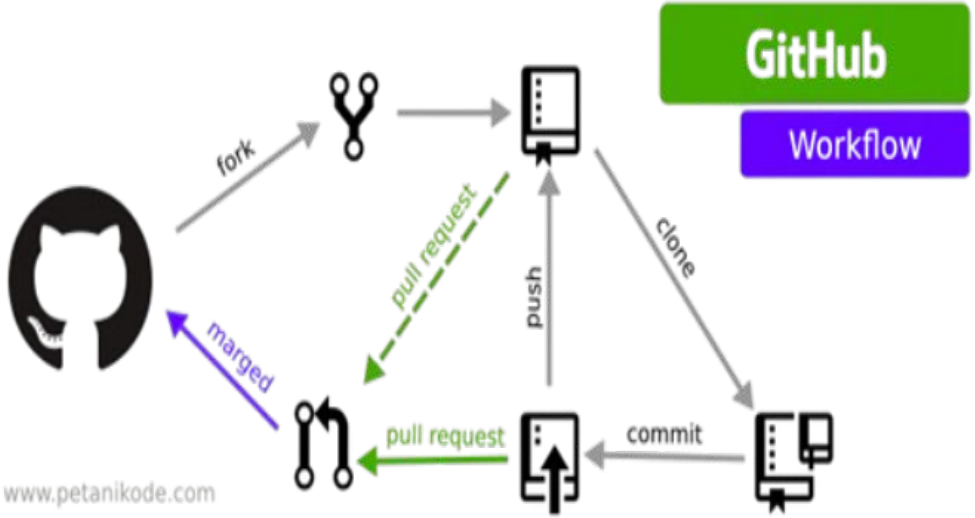
2. Control de versiones - Estructura del sistema

Funciona mediante un modelo **cliente/servidor**, donde existe un **repositorio remoto** y **copias locales** del proyecto en cada ordenador, que **se sincronizan entre sí**.



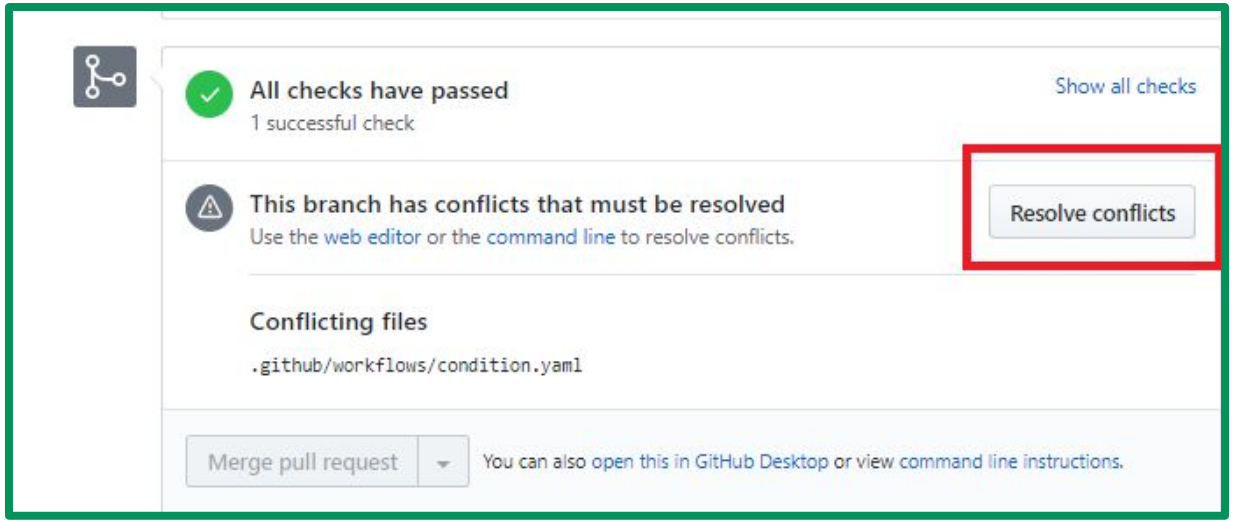
2. Control de versiones - Operaciones básicas

El desarrollador **descarga** el proyecto, lo **modifica** en local y, una vez terminadas, **sube** los cambios al repositorio para que el resto del equipo pueda acceder a ellos.



2. Control de versiones - Operaciones avanzadas

Además de las operaciones básicas, se pueden **comparar versiones**, **recuperar código antiguo** y **resolver conflictos** cuando varios usuarios modifican el mismo archivo.



3. Documentación del software - Uso de comentarios

Los **comentarios** se utilizan para **explicar el código cuando puede resultar complejo**, ayudando a su comprensión y mantenimiento a largo plazo.

```java
public class Nose {

    // Método que suma dos números enteros
    public int sumar(int a, int b) {
        return a + b;
    }

    // Método que resta dos números enteros
    public int restar(int a, int b) {
        return a - b;

    /*
        comentario de varias
        líneas
    */

    // comentario de una sola línea
    }
}
```

3. Documentación del software - Buenas prácticas

Un buen comentario debe ser **claro, útil y estar actualizado**. **No** es recomendable **comentar cosas evidentes ni saturar el código** con explicaciones innecesarias.

```
public int multiplicar(int a, int b) {
// Multiplica a por b
return a * b;
}
```

Comentario innecesario, ya que el código se entiende solo.

Los comentarios aportan **información relevante** que no se deduce directamente del código.

```
public int dividir(int a, int b) {
    // Devuelve el resultado de la división.
    // Es necesario comprobar que el divisor no sea cero.
    return a / b;
}
```

3. Documentación del software - Documentación automática

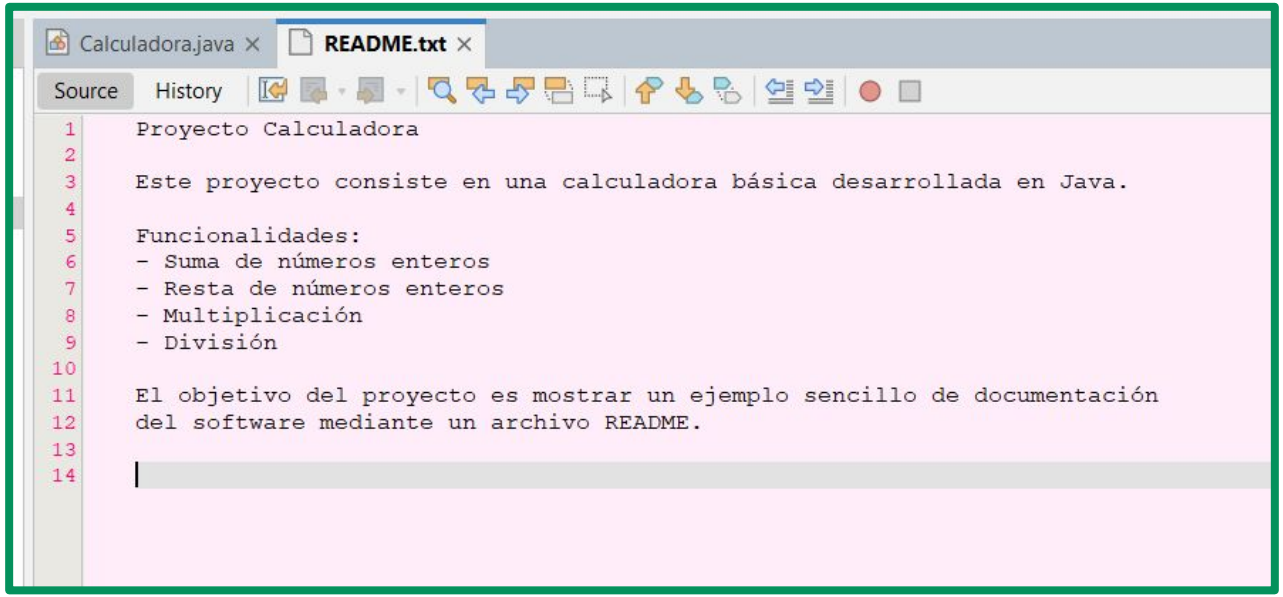
Existen herramientas que **generan documentación automáticamente a partir del código fuente**, facilitando la creación de manuales técnicos.

```java
/**
 * Clase que representa una calculadora básica.
 * Permite realizar operaciones matemáticas simples.
 */
public class Calculadora {

    /**
     * Suma dos números enteros.
     *
     * @param a Primer número
     * @param b Segundo número
     * @return Resultado de la suma
     */
    public int sumar(int a, int b) {
        return a + b;
    }

    /**
     * Resta dos números enteros.
     *
     * @param a Primer número
     * @param b Segundo número
     * @return Resultado de la resta
     */
    public int restar(int a, int b) {
        return a - b;
    }
}
```

En este ejemplo se muestra el uso de comentarios **JavaDoc**, base para la generación automática de documentación.

3. Documentación del software - Alternativas

Además de los comentarios en el código, se puede documentar el software mediante archivos **README**, **wikis** de proyecto o diagramas **UML**.



PREGUNTAS

PREGUNTAS

1. ¿Cuál es la definición correcta de refactorización?
   a) Añadir nuevos aspectos al programa
   b) Corregir errores que ve el usuario al usar el programa
   c) Reorganizar y mejorar el código fuente ✅
   d) Optimizar rendimiento cambiando el resultado final

2. ¿Cuál de los siguientes IDEs usa el comando Alt + Shift + R para renombrar?
   a) IntelliJ
   b) NetBeans
   c) VS Code
   d) Eclipse ✅

PREGUNTAS

3. ¿Qué característica de las pruebas unitarias es falsa?
   a) Son rápidas
   b) Prueban con métodos más pequeños que el código funcione
   c) Toman mucho espacio en el código ✅
   d) Sirven como apoyo

4. ¿Cómo funciona el control de versiones?
   a) Mediante un modelo cliente/servidor
   b) Con un repositorio y pruebas locales
   c) Manteniendo un historial completo del proyecto
   d) Todas las respuestas son correctas ✅

PREGUNTAS

5. ¿Qué significa que un comentario sea innecesario?
   a) Que aporte información relevante
   b) Que sea evidente ✅
   c) Que esté actualizado
   d) Que ayude a la compresión de código complejo